

## Section 76 Editorial — Suffix Arrays And LCP Structure

Each solution starts from the uncompressed object, identifies the bottleneck, and states the invariant that justifies the compressed structure.

### A. Suffix Order And LCP

#### How to find it

Replace suffix strings by ranks of power-of-two prefixes. Kasai then reuses all but the first character of the previous comparison.

#### Correctness

The doubling invariant proves the final ranks are the exact suffix order. Kasai compares the suffix preceding each suffix in that order, so every requested adjacent LCP is produced.

#### Complexity

$O(n \log^2 n)$  time and  $O(n)$  memory with comparison sorting.

#### Trace, pseudocode, and traps

For **banana**, first-character ranks tie positions 1,3,5; successive pair ranks separate suffix **a**, **ana**, and **anana**. In each round sort positions by  $(\text{rank}[i], \text{rank}[i+\text{length}])$ , assign equal new ranks to equal pairs, and double **length**. Kasai then scans text positions, compares only with the previous suffix in final order, and reuses  $\max(h-1, 0)$  characters.

Do not build copied suffix strings, confuse text order with suffix-array order, or omit the smaller sentinel rank for a block ending past **n**. Test length one, all equal characters, and a string whose suffix is a prefix of another.

#### Reference C++

```
#include <bits/stdc++.h>
using namespace std;
vector<int> suffix_array(const string &s) {
    int n = s.size();
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i)
        r[i] = (unsigned char)s[i];
    for (int k = 1; k <= 2; k *= 2) {
        sort(sa.begin(), sa.end(), [&](int a, int b) {
            return pair{r[a], a + k < n ? r[a + k] : -1} <
                pair{r[b], b + k < n ? r[b + k] : -1};
        });
        nr[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            nr[b] = nr[a] + (pair{r[a], a + k < n ? r[a + k] : -1} <
                pair{r[b], b + k < n ? r[b + k] : -1});
        }
    }
    r = nr;
}
```

```

        if (r[sa.back()] == n - 1)
            break;
    }
    return sa;
}

vector<int> lcp_array(const string &s, const vector<int> &sa) {
    int n = s.size(), h = 0;
    vector<int> rank(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; ++i)
        rank[sa[i]] = i;
    for (int i = 0; i < n; ++i) {
        int q = rank[i];
        if (!q)
            continue;
        int j = sa[q - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h])
            ++h;
        lcp[q - 1] = h;
        if (h)
            --h;
    }
    return lcp;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    auto sa = suffix_array(s), lcp = lcp_array(s, sa);
    for (int i = 0; i < (int)sa.size(); ++i)
        cout << sa[i] + 1 << " \n"[i + 1 == (int)sa.size()];
    for (int i = 0; i < (int)lcp.size(); ++i)
        cout << lcp[i] << " \n"[i + 1 == (int)lcp.size()];
    if (lcp.empty())
        cout << '\n';
}

```

## Reference Python

```

def suffix_array(s):
    n = len(s)
    sa = list(range(n))
    rank = list(map(ord, s))
    step = 1
    while True:
        sa.sort(
            key=lambda i: (rank[i], rank[i + step] if i + step < n else -1)
        )
        new_rank = [0] * n
        for j in range(1, n):
            a, b = sa[j - 1], sa[j]
            left = (rank[a], rank[a + step] if a + step < n else -1)
            right = (rank[b], rank[b + step] if b + step < n else -1)
            new_rank[b] = new_rank[a] + (left < right)
        rank = new_rank
        if rank[sa[-1]] == n - 1:
            return sa
        step *= 2

```

```

def lcp_array(s, sa):
    n = len(s)
    rank = [0] * n
    for i, start in enumerate(sa):
        rank[start] = i
    lcp = [0] * (n - 1)
    height = 0
    for i in range(n):
        place = rank[i]
        if place == 0:
            continue
        j = sa[place - 1]
        while (
            i + height < n
            and j + height < n
            and s[i + height] == s[j + height]
        ):
            height += 1
        lcp[place - 1] = height
        height = max(0, height - 1)
    return lcp

def main():
    import sys

    s = sys.stdin.buffer.readline().decode().strip()
    sa = suffix_array(s)
    print(*(start + 1 for start in sa))
    print(*lcp_array(s, sa))

if __name__ == "__main__":
    main()

```

## B. Repeated At Least K Times

### How to find it

Sort the occurrence starts indirectly as suffixes. A length shared by  $k$  consecutive suffixes is exactly the minimum of the intervening LCP values.

### Correctness

Every substring with  $k$  occurrences yields a block whose LCP minimum is at least its length. Conversely, a block minimum  $L$  makes the first  $L$  characters equal in all suffixes of that block.

### Complexity

$O(n \log^2 n)$  construction and  $O(n)$  deque processing.

### Worked window and traps

For four consecutive suffixes with LCP values 5,3,4, their shared prefix is length 3. A  $k$ -suffix block contains  $k-1$  adjacent LCP entries, so the deque window has length  $k-1$ , not  $k$ . For  $k=1$ , the entire string is a valid one-occurrence substring and must be handled separately.

Maintain increasing LCP values in the deque and remove indices leaving the window before reading its front. Taking the maximum LCP instead of the minimum confuses one good adjacent pair with a prefix common to the whole block.

## Reference C++

```
#include <bits/stdc++.h>
using namespace std;
vector<int> suffix_array(const string &s) {
    int n = s.size();
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i)
        r[i] = (unsigned char)s[i];
    for (int k = 1;; k *= 2) {
        sort(sa.begin(), sa.end(), [&](int a, int b) {
            return pair{r[a], a + k < n ? r[a + k] : -1} <
                pair{r[b], b + k < n ? r[b + k] : -1};
        });
        nr[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            nr[b] = nr[a] + (pair{r[a], a + k < n ? r[a + k] : -1} <
                pair{r[b], b + k < n ? r[b + k] : -1});
        }
        r = nr;
        if (r[sa.back()] == n - 1)
            break;
    }
    return sa;
}

vector<int> lcp_array(const string &s, const vector<int> &sa) {
    int n = s.size(), h = 0;
    vector<int> rank(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; ++i)
        rank[sa[i]] = i;
    for (int i = 0; i < n; ++i) {
        int q = rank[i];
        if (!q)
            continue;
        int j = sa[q - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h])
            ++h;
        lcp[q - 1] = h;
        if (h)
            --h;
    }
    return lcp;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    int k;
    cin >> s >> k;
    int n = s.size();
    if (k == 1) {
        cout << n << '\n';
        return 0;
    }
    auto sa = suffix_array(s), lcp = lcp_array(s, sa);
    deque<int> q;
    int ans = 0, w = k - 1;
    for (int i = 0; i < n - 1; ++i) {
        while (!q.empty() && lcp[q.back()] >= lcp[i])
            q.pop_back();
    }
```

```

    q.push_back(i);
    while (q.front() <= i - w)
        q.pop_front();
    if (i + 1 >= w)
        ans = max(ans, lcp[q.front()]);
}
cout << ans << '\n';
}

```

## Reference Python

```

def suffix_array(s):
    n = len(s)
    sa = list(range(n))
    rank = list(map(ord, s))
    step = 1
    while True:
        sa.sort(
            key=lambda i: (rank[i], rank[i + step] if i + step < n else -1)
        )
        new_rank = [0] * n
        for j in range(1, n):
            a, b = sa[j - 1], sa[j]
            left = (rank[a], rank[a + step] if a + step < n else -1)
            right = (rank[b], rank[b + step] if b + step < n else -1)
            new_rank[b] = new_rank[a] + (left < right)
        rank = new_rank
        if rank[sa[-1]] == n - 1:
            return sa
        step *= 2

def lcp_array(s, sa):
    n = len(s)
    rank = [0] * n
    for i, start in enumerate(sa):
        rank[start] = i
    lcp = [0] * (n - 1)
    height = 0
    for i in range(n):
        place = rank[i]
        if place == 0:
            continue
        j = sa[place - 1]
        while (
            i + height < n
            and j + height < n
            and s[i + height] == s[j + height]
        ):
            height += 1
        lcp[place - 1] = height
        height = max(0, height - 1)
    return lcp

def main():
    import collections
    import sys

    s, raw_k = sys.stdin.buffer.read().split()
    s, k = s.decode(), int(raw_k)

```

```

if k == 1:
    print(len(s))
    return
lcp = lcp_array(s, suffix_array(s))
window = k - 1
queue = collections.deque()
answer = 0
for i, value in enumerate(lcp):
    while queue and lcp[queue[-1]] >= value:
        queue.pop()
    queue.append(i)
    while queue[0] <= i - window:
        queue.popleft()
    if i + 1 >= window:
        answer = max(answer, lcp[queue[0]])
print(answer)

if __name__ == "__main__":
    main()

```

## C. Pattern Occurrence Queries

### How to find it

All suffixes prefixed by a fixed pattern are consecutive. Binary-search the first suffix not smaller than the pattern and the first suffix strictly greater under prefix comparison.

### Correctness

The comparator partitions the sorted suffixes into smaller, matching, and greater groups. Their middle interval contains exactly all occurrence starts.

### Complexity

$O(n \log^2 n + \sum |p| \log n)$  time and  $O(n)$  memory.

### Comparator contract and tests

The lower search asks whether a suffix is lexicographically smaller than the pattern. The upper search asks whether it still begins with the complete pattern. Compare directly in the source text; if the pattern ends first, it is a match, while a suffix ending first is smaller.

Test a pattern equal to the entire text, longer than every suffix, absent between two suffixes, and repeated at overlapping starts. Returning suffix- array ranks instead of original start positions is another common mistake.

### Reference C++

```

#include <bits/stdc++.h>
using namespace std;
vector<int> suffix_array(const string &s) {
    int n = s.size();
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i)
        r[i] = (unsigned char)s[i];
    for (int k = 1; k <= 2) {
        sort(sa.begin(), sa.end(), [&](int a, int b) {
            return pair{r[a], a + k < n ? r[a + k] : -1} <

```

```

        pair{r[b], b + k < n ? r[b + k] : -1};
    });
    nr[sa[0]] = 0;
    for (int i = 1; i < n; ++i) {
        int a = sa[i - 1], b = sa[i];
        nr[b] = nr[a] + (pair{r[a], a + k < n ? r[a + k] : -1} <
                        pair{r[b], b + k < n ? r[b + k] : -1});
    }
    r = nr;
    if (r[sa.back()] == n - 1)
        break;
}
return sa;
}

vector<int> lcp_array(const string &s, const vector<int> &sa) {
    int n = s.size(), h = 0;
    vector<int> rank(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; ++i)
        rank[sa[i]] = i;
    for (int i = 0; i < n; ++i) {
        int q = rank[i];
        if (!q)
            continue;
        int j = sa[q - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h])
            ++h;
        lcp[q - 1] = h;
        if (h)
            --h;
    }
    return lcp;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    int q;
    cin >> s >> q;
    auto sa = suffix_array(s);
    while (q--) {
        string p;
        cin >> p;
        auto cmp = [&](int at) {
            int z = s.compare(at, p.size(), p);
            return z < 0 ? -1 : z > 0 ? 1 : 0;
        };
        int l = 0, r = sa.size();
        while (l < r) {
            int m = (l + r) / 2;
            if (cmp(sa[m]) < 0)
                l = m + 1;
            else
                r = m;
        }
        int first = l;
        l = 0;
        r = sa.size();
        while (l < r) {
            int m = (l + r) / 2;
            if (cmp(sa[m]) <= 0)
                l = m + 1;
            else

```

```

        r = m;
    }
    cout << l - first << '\n';
}
}

```

## Reference Python

```

def suffix_array(s):
    n = len(s)
    sa = list(range(n))
    rank = list(map(ord, s))
    step = 1
    while True:
        sa.sort(
            key=lambda i: (rank[i], rank[i + step] if i + step < n else -1)
        )
        new_rank = [0] * n
        for j in range(1, n):
            a, b = sa[j - 1], sa[j]
            left = (rank[a], rank[a + step] if a + step < n else -1)
            right = (rank[b], rank[b + step] if b + step < n else -1)
            new_rank[b] = new_rank[a] + (left < right)
        rank = new_rank
        if rank[sa[-1]] == n - 1:
            return sa
        step *= 2

def lcp_array(s, sa):
    n = len(s)
    rank = [0] * n
    for i, start in enumerate(sa):
        rank[start] = i
    lcp = [0] * (n - 1)
    height = 0
    for i in range(n):
        place = rank[i]
        if place == 0:
            continue
        j = sa[place - 1]
        while (
            i + height < n
            and j + height < n
            and s[i + height] == s[j + height]
        ):
            height += 1
        lcp[place - 1] = height
        height = max(0, height - 1)
    return lcp

def main():
    import sys

    data = sys.stdin.buffer.read().split()
    s = data[0].decode()
    sa = suffix_array(s)
    output = []
    for raw in data[2:]:
        pattern = raw.decode()

```

```

def compare(start):
    piece = s[start : start + len(pattern)]
    return (piece > pattern) - (piece < pattern)

left, right = 0, len(sa)
while left < right:
    middle = (left + right) // 2
    if compare(sa[middle]) < 0:
        left = middle + 1
    else:
        right = middle
first = left
left, right = 0, len(sa)
while left < right:
    middle = (left + right) // 2
    if compare(sa[middle]) <= 0:
        left = middle + 1
    else:
        right = middle
output.append(str(left - first))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

## D. Disjoint Repeated Substring

### How to find it

Binary-search the length. LCP values at least that length join maximal groups of suffixes sharing the candidate prefix; track the minimum and maximum starting positions in each group.

### Correctness

Two equal occurrences are disjoint exactly when their starts differ by at least the tested length. The scan examines precisely every maximal equal-prefix group, so feasibility is exact and monotone.

### Complexity

$O(n \log^2 n + n \log n)$  time and  $O(n)$  memory.

### Detailed feasibility proof and pseudocode

For a tested length  $L$ , scan the suffix array and start a new group whenever the intervening LCP is below  $L$ . Within one group, maintain the minimum and maximum suffix start. The group is feasible exactly when `maximum_start - minimum_start >= L`; then the two equal length- $L$  occurrences do not overlap.

```

feasible(L):
    begin first group with min_start=max_start=sa[0]
    for each adjacent LCP:
        if lcp >= L: extend extrema with next suffix
        else: test old group and start a new one
    test final group

```

If  $L$  is feasible, every shorter length is feasible using the same starts, so binary search is valid. Check the final group after the loop, use  $\geq L$  rather than  $> L$ , and test occurrences that touch end-to-start exactly.

## Reference C++

```
#include <bits/stdc++.h>
using namespace std;
vector<int> suffix_array(const string &s) {
    int n = s.size();
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i)
        r[i] = (unsigned char)s[i];
    for (int k = 1;; k *= 2) {
        sort(sa.begin(), sa.end(), [&](int a, int b) {
            return pair{r[a], a + k < n ? r[a + k] : -1} <
                pair{r[b], b + k < n ? r[b + k] : -1};
        });
        nr[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            nr[b] = nr[a] + (pair{r[a], a + k < n ? r[a + k] : -1} <
                pair{r[b], b + k < n ? r[b + k] : -1});
        }
        r = nr;
        if (r[sa.back()] == n - 1)
            break;
    }
    return sa;
}

vector<int> lcp_array(const string &s, const vector<int> &sa) {
    int n = s.size(), h = 0;
    vector<int> rank(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; ++i)
        rank[sa[i]] = i;
    for (int i = 0; i < n; ++i) {
        int q = rank[i];
        if (!q)
            continue;
        int j = sa[q - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h])
            ++h;
        lcp[q - 1] = h;
        if (h)
            --h;
    }
    return lcp;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size();
    auto sa = suffix_array(s), lcp = lcp_array(s, sa);
    auto ok = [&](int need) {
        int lo = sa[0], hi = sa[0];
        for (int i = 0; i < n - 1; ++i) {
            if (lcp[i] < need)
                lo = hi = sa[i + 1];
            else {
                lo = min(lo, sa[i + 1]);
                hi = max(hi, sa[i + 1]);
                if (hi - lo >= need)
                    return true;
            }
        }
        return false;
    };
    for (int i = 0; i < n; ++i)
        if (ok(i))
            cout << i << endl;
}
```

```

    }
}
return false;
};
int l = 0, r = n / 2 + 1;
while (r - l > 1) {
    int m = (l + r) / 2;
    (ok(m) ? l : r) = m;
}
cout << l << '\n';
}

```

## Reference Python

```

def suffix_array(s):
    n = len(s)
    sa = list(range(n))
    rank = list(map(ord, s))
    step = 1
    while True:
        sa.sort(
            key=lambda i: (rank[i], rank[i + step] if i + step < n else -1)
        )
        new_rank = [0] * n
        for j in range(1, n):
            a, b = sa[j - 1], sa[j]
            left = (rank[a], rank[a + step] if a + step < n else -1)
            right = (rank[b], rank[b + step] if b + step < n else -1)
            new_rank[b] = new_rank[a] + (left < right)
        rank = new_rank
        if rank[sa[-1]] == n - 1:
            return sa
        step *= 2

def lcp_array(s, sa):
    n = len(s)
    rank = [0] * n
    for i, start in enumerate(sa):
        rank[start] = i
    lcp = [0] * (n - 1)
    height = 0
    for i in range(n):
        place = rank[i]
        if place == 0:
            continue
        j = sa[place - 1]
        while (
            i + height < n
            and j + height < n
            and s[i + height] == s[j + height]
        ):
            height += 1
        lcp[place - 1] = height
        height = max(0, height - 1)
    return lcp

def main():
    import sys

```

```

s = sys.stdin.buffer.readline().decode().strip()
sa = suffix_array(s)
lcp = lcp_array(s, sa)

def possible(length):
    low = high = sa[0]
    for i, common in enumerate(lcp):
        if common < length:
            low = high = sa[i + 1]
        else:
            low = min(low, sa[i + 1])
            high = max(high, sa[i + 1])
            if high - low >= length:
                return True
    return False

low, high = 0, len(s) // 2 + 1
while high - low > 1:
    middle = (low + high) // 2
    if possible(middle):
        low = middle
    else:
        high = middle
print(low)

if __name__ == "__main__":
    main()

```